

Digital Signal Processing Lab
Final Project Report
Voice Conversion using WORLD Vocoder

Saad Zubairi
shz2020

December 19th, 2025

Abstract

The final project implements a real-time voice conversion system that shifts pitch and formants of Speaker A to match Speaker B using the WORLD Vocoder. Calibration records both speakers, extracts median pitch and formant statistics, and computes scaling values for live conversion. The live pipeline streams audio from the mic, processes blocks in a background thread, and synthesizes transformed speech. The implementation is modular and thread-safe, separating audio I/O, DSP, and UI control.

1 Introduction

This voice conversion project tries to change a speaker's pitch and timbre so it sounds more like another person, but still keeps the speech clear enough to understand. Basically, the idea is to take Speaker A and make them sound like Speaker B in real time, by shifting their fundamental frequency (F0) and also the formant structure, and it does this with a calibration step first to figure out what each speaker sounds like, then it runs live and keeps processing audio blocks. The software is built to be modular and uses thread-safe streaming, and you get direct control over pitch and formant shifts, so it's pretty flexible, and user can experiment with different voices.

2 Background and why WORLD

The WORLD vocoder structure represents speech using three components:

- **The fundamental frequency (F0):** The rate of vocal fold vibration, perceived as pitch.
- **Spectral envelope:** Represents the resonant characteristics of the vocal tract, shaping the timbre.
- **Aperiodicity measure:** Captures noise components in the speech signal (could be breathiness or the s,f,and h sounds).

For this project, we use the following for the analysis that works within the WORLD vocoder setup:

- **DIO and StoneMask (Pitch Estimation):** DIO quickly estimates the fundamental frequency (F0) by searching for periodicity in the waveform. StoneMask refines this F0 track using spectral information, improving accuracy in voiced regions.
- **CheapTrick (Spectral Envelope Estimation):** CheapTrick extracts the spectral envelope, modeling vocal tract resonances. It uses a smoothed log spectrum for efficient and high-quality envelope estimation, capturing timbre and formant structure.
- **D4C (Aperiodicity Estimation):** D4C measures aperiodicity for each frame, distinguishing voiced and unvoiced sounds. This helps model breathiness and fricatives, making synthesis sound more natural.

After analysis, the system modifies F0 and the spectral envelope to achieve the desired voice conversion. Pitch shifting is implemented by scaling voiced F0 values, while formant shifting is implemented by remapping the spectral envelope bins.

3 System Overview

The project is divided into the following modules:

- `app.py`: main application, calibration workflow, live controls, and worker thread.
- `audio/recorder.py`: audio recording and playback for calibration.
- `audio/pyaudio_io.py`: real-time audio streaming using a callback.
- `audio/ringbuffer.py`: thread-safe ring buffers for audio blocks.
- `dsp/world_vocoder.py`: WORLD analysis, modification, and synthesis.
- `dsp/voice_profile.py`: extraction and storage of speaker profiles.

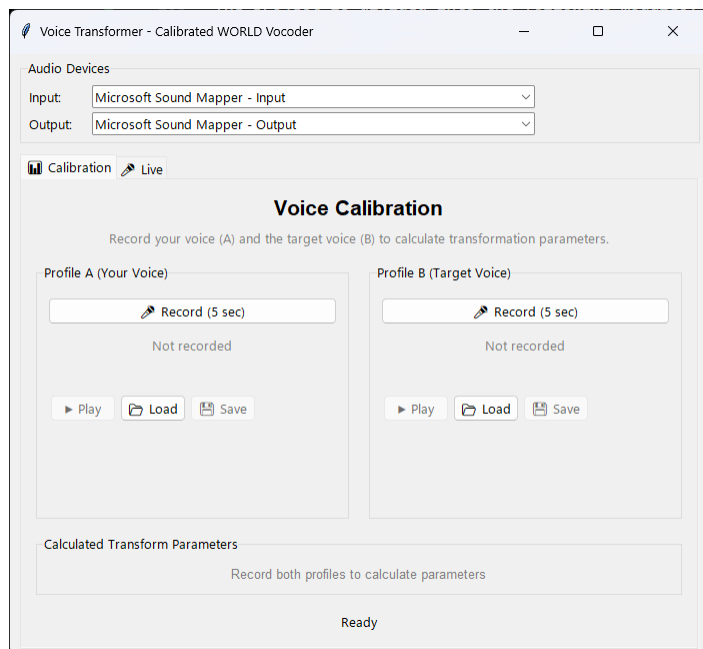
Of course, for debugging purposes and keeping the parameters context, a utility package was also used following some standard boilerplate python practices (not directly relevant to the DSP implementation).

4 Implementation

a) Voice Calibration and Parameter Mapping

During calibration, the app quickly records two five-second samples, one for Profile A and one for Profile B, and then automatically extracts the median F0 and formant values from both. It actually computes the pitch shift by comparing the median F0s (in semitones), and the formant shift by taking the ratio of the average F1 and F2 medians. The values, of course, are clipped so they don't go out of range, but sometimes the results can still be a bit off if the recordings aren't clean or if someone talks unusually during calibration.

Here's what the screen looks like during the initial calibration step:



```

1 def calculate_transform_params(profile_a: VoiceProfile, profile_b: VoiceProfile)
  -> dict:
2     """Calculate pitch and formant shift from two voice profiles."""
3     if profile_a.f0_median_hz > 0 and profile_b.f0_median_hz > 0:
4         f0_ratio = profile_b.f0_median_hz / profile_a.f0_median_hz
5         pitch_shift = 12 * np.log2(f0_ratio)
6     else:
7         pitch_shift = 0.0
8
9     formants_a = []
10    formants_b = []
11
12    if profile_a.formant_f1_median > 0 and profile_b.formant_f1_median > 0:
13        formants_a.append(profile_a.formant_f1_median)
14        formants_b.append(profile_b.formant_f1_median)
15
16    if profile_a.formant_f2_median > 0 and profile_b.formant_f2_median > 0:
17        formants_a.append(profile_a.formant_f2_median)
18        formants_b.append(profile_b.formant_f2_median)
19
20    if formants_a and formants_b:
21        avg_formant_a = np.mean(formants_a)
22        avg_formant_b = np.mean(formants_b)
23        formant_shift = avg_formant_b / avg_formant_a
24    else:
25        formant_shift = 1.0
26
27    pitch_shift = np.clip(pitch_shift, -12, 12)
28    formant_shift = np.clip(formant_shift, 0.5, 2.0)
29
30    return {
31        'pitch_shift': float(pitch_shift),
32        'formant_shift': float(formant_shift)
33    }

```

Snippet 1: Pitch and formant mapping from two voice profiles (app.py).

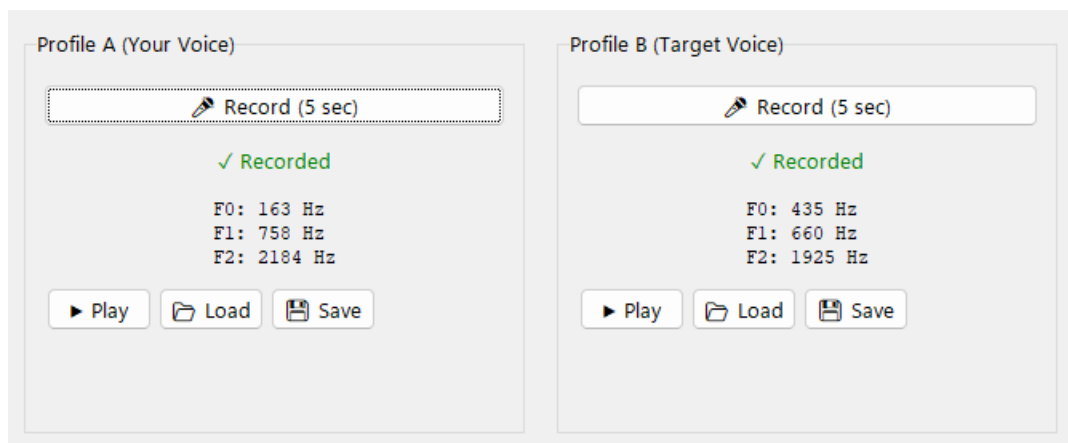
b) WORLD Analysis and Synthesis

The WORLD analysis and synthesis, which are performed by the `WorldVocoder` class using `PyWorld`, are typically carried out in a way that the F0, spectral envelope, and aperiodicity are automatically extracted during the analysis stage, and then the synthesis stage is executed so that audio is reconstructed from those parameters.

```
1  def analyze(self, audio: np.ndarray) -> WorldParams:
2      """Analyze audio to extract WORLD parameters."""
3      audio = audio.astype(np.float64)
4
5      f0, t = pw.dio(audio, self.sample_rate, frame_period=self.frame_period)
6      f0 = pw.stonemask(audio, f0, t, self.sample_rate)
7
8      sp = pw.cheaptrick(audio, f0, t, self.sample_rate)
9      ap = pw.d4c(audio, f0, t, self.sample_rate)
10
11     return WorldParams(
12         f0=f0,
13         sp=sp,
14         ap=ap,
15         sample_rate=self.sample_rate,
16         frame_period=self.frame_period
17     )
18
19     def synthesize(self, params: WorldParams) -> np.ndarray:
20         """Synthesize audio from WORLD parameters."""
21         audio = pw.synthesize(
22             params.f0,
23             params.sp,
24             params.ap,
25             params.sample_rate,
26             params.frame_period
27         )
28         return audio.astype(np.float32)
```

Snippet 2: WORLD analysis and synthesis (`dsp/world_vocoder.py`).

After recording the voices, the calibration screen displays the extracted statistics for both profiles, along with the computed pitch and formant shifts:



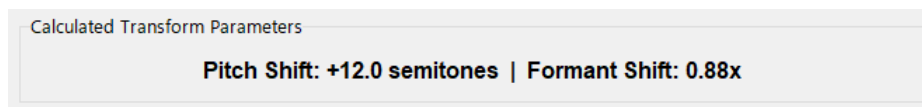
c) Pitch and Formant Modification

Pitch shifting is performed by scaling the voiced F0 values by a ratio that is computed from the semitone shift, and formant shifting is achieved by remapping the spectral envelope bins according to the desired ratio, so both operations are applied in a way that preserves the remaining WORLD parameters, and a new `WorldParams` object is consequently returned, effectively ensuring that the transformations are applied consistently and efficiently.

```
1  def shift_pitch(self, params: WorldParams, semitones: float) -> WorldParams:
2      """Shift pitch by semitones."""
3      ratio = 2.0 ** (semitones / 12.0)
4
5      new_f0 = params.f0.copy()
6      voiced = new_f0 > 0
7      new_f0[voiced] = new_f0[voiced] * ratio
8
9      return WorldParams(
10         f0=new_f0,
11         sp=params.sp,
12         ap=params.ap,
13         sample_rate=params.sample_rate,
14         frame_period=params.frame_period
15     )
16
17 def shift_formants(self, params: WorldParams, shift_ratio: float) ->
WorldParams:
18     """Shift formants (spectral envelope)."""
19     sp = params.sp
20     new_sp = np.zeros_like(sp)
21
22     freq_bins = sp.shape[1]
23
24     for i in range(sp.shape[0]):
25         for j in range(freq_bins):
26             src_idx = int(j / shift_ratio)
27             if 0 <= src_idx < freq_bins:
28                 new_sp[i, j] = sp[i, src_idx]
29             else:
30                 new_sp[i, j] = sp[i, -1]
31
32     return WorldParams(
33         f0=params.f0,
34         sp=new_sp,
35         ap=params.ap,
36         sample_rate=params.sample_rate,
37         frame_period=params.frame_period
38     )
```

Snippet 3: Pitch and formant modification (`dsp/world_vocoder.py`).

The calculated pitch and formant shifts are displayed on the calibration screen after recording as follows:



d) Real-Time Processing and Thread Safety

Live processing runs in a background thread. The worker waits for a full block in the input ring buffer, processes the block with the WORLD vocoder, and pushes the output to the output ring buffer. In my implementation, I found that a block size of 1024 samples works well for low latency and stable processing.

```
1  def run(self):
2      self._running = True
3
4      while self._running:
5          if self._paused:
6              time.sleep(0.01)
7              continue
8
9          available = self.input_buffer.count
10
11         if available >= self.block_size:
12             samples = self.input_buffer.pop(self.block_size)
13
14             if samples is not None:
15                 with self._lock:
16                     self._input_level_db = calculate_level_db(samples)
17
18                 if self._bypass:
19                     output = samples
20                 else:
21                     output = self.vocoder.process(samples)
22                     with self._lock:
23                         self._process_time_ms = self.vocoder.process_time_ms
24
25                 with self._lock:
26                     self._output_level_db = calculate_level_db(output)
27
28                 self.output_buffer.push(output)
29             else:
30                 time.sleep(0.005)
```

Snippet 4: Background processing thread (app.py).

Thread-safe ring buffers are used between the audio callback and the worker thread. Both read and write operations are protected by a lock.

```
1  def push(self, data: np.ndarray) -> int:
2      """Push samples into the buffer."""
3      n = len(data)
4
5      with self._lock:
6          available = self._capacity - self._count
7          n_to_write = min(n, available)
8
9          if n_to_write == 0:
10             return 0
11
12         end_space = self._capacity - self._write_idx
13
14         if n_to_write <= end_space:
15             self._buffer[self._write_idx:self._write_idx + n_to_write] = data[:n_to_write]
16         else:
```

```

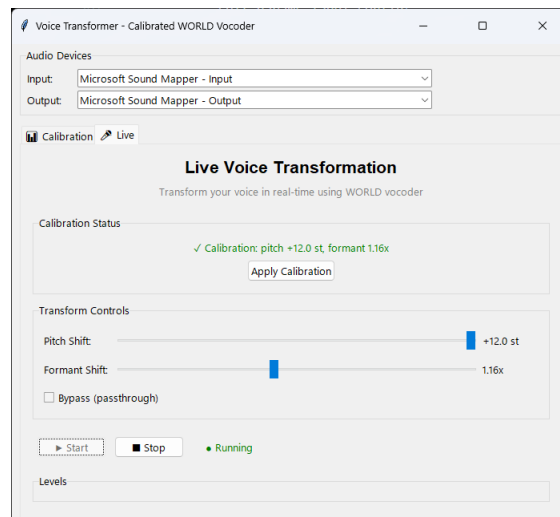
17         self._buffer[self._write_idx:] = data[:end_space]
18         remaining = n_to_write - end_space
19         self._buffer[:remaining] = data[end_space:n_to_write]
20
21         self._write_idx = (self._write_idx + n_to_write) % self._capacity
22         self._count += n_to_write
23
24     return n_to_write
25
26 def pop(self, n: int) -> Optional[np.ndarray]:
27     """Pop samples from the buffer."""
28     with self._lock:
29         if self._count < n:
30             return None
31
32         result = np.zeros(n, dtype=self._dtype)
33         end_space = self._capacity - self._read_idx
34
35         if n <= end_space:
36             result[:] = self._buffer[self._read_idx:self._read_idx + n]
37         else:
38             result[:end_space] = self._buffer[self._read_idx:]
39             remaining = n - end_space
40             result[end_space:] = self._buffer[:remaining]
41
42         self._read_idx = (self._read_idx + n) % self._capacity
43         self._count -= n
44
45     return result

```

Snippet 5: Thread-safe ring buffer write and read (audio/ringbuffer.py).

5 Realtime conversion interface

The live tab provides controls to start/stop processing, adjust pitch and formant shifts, and toggle bypass mode. We can apply the extracted parameters which we got from the calibration step, or also manually adjust them with sliders. The converted voice is then relayed through the speakers in real time.



6 Known Issues and Limitations

During testing and development, several limitations became apparent based on how the system behaved in practice.

- **Sensitivity during calibration.** In my testing, I noticed that the calibration step is fairly sensitive to how the speaker talks during the five-second recording. If the speaker varies their pitch too much, speaks very softly, or includes long pauses, the extracted median F0 and formant values can be skewed. When this happens, the resulting pitch or formant shift can sound unnatural during live conversion. I'd recommend adapting a robotic and steady speaking style for the best results.
- **Simplified formant shifting.** The current formant-shifting approach uses a single global scaling factor derived from average F1 and F2 values. In practice, this works reasonably well for small shifts, but for larger shifts I found that the converted voice can sound slightly muffled or distorted, likely because the spectral envelope is being remapped uniformly rather than preserving detailed formant structure across frequencies.
- **Latency versus stability tradeoff.** While experimenting with different block sizes, I found that smaller blocks reduce latency but significantly increase CPU load, frequently causing audio dropouts that makes the audio very choppy. A block size of 1024 samples ended up being a reasonable compromise, but this choice is hardware-dependent.
- **Limited robustness across speakers.** The system works best when converting between two speakers with relatively similar speaking styles. In my testing, applying calibration parameters to voices with very different pitch ranges or vocal characteristics often produced less convincing results, indicating limited generalization beyond the calibrated pair.

7 Future Work

I am currently trying to minify the system to run on an embedded platform, like an STM32 board. The plan is to implement a simplified version of this pipeline on an STM microcontroller with a UART interface for microphone input. The board would act as a USB or serial audio device that captures audio, performs real-time pitch and spectral modification, and relays the converted signal back to a host system. A sort of faux audio converter microphone that can be calibrated to sound like different speakers.

8 Addendum

This project is structured with the following modular, object-oriented utilities for better future scaling and clarity of implementation:

- **Custom Logging:** A custom logging system (`utils/logging_utils.py`) which provides structured logs for audio, DSP, and UI, following Python best practices.
- **Configuration Management:** All audio and runtime parameters are centralized in `utils/config.py` using dataclasses, enabling easy adjustment and consistent access throughout the project.
- **Thread Safe Ring Buffer:** The `audio/ringbuffer.py` module implements a lock-protected circular buffer for realtime audio streaming, giving safe concurrent access between threads.
- **PyAudio Boilerplate:** The `audio/pyaudio_io.py` module encapsulates device management, stream setup, and callback logic, abstracting away low-level PyAudio details.
- **Recorder Module:** `audio/recorder.py` provides a simple, reusable class for background audio recording and playback, using OOP patterns and dataclasses for state management.

These modules are built with class boundaries, encapsulation, and minimal side effects, following common Python OOP and modularity standards.

Dependencies: Dependencies required are as follows: (in `requirements.txt`):

- `numpy (>=1.20.0)`
- `scipy (>=1.7.0)`
- `pyaudio (>=0.2.11)`
- `pyworld (>=0.3.0)`

References

1. M. Morise, F. Yokomori, and K. Ozawa: WORLD: a vocoder-based high-quality speech synthesis system for real-time applications, IEICE transactions on information and systems, vol. E99-D, no. 7, pp. 1877-1884, 2016. https://www.jstage.jst.go.jp/article/transinf/E99.D/7/E99.D_2015EDP7457/_article
2. M. Morise: D4C, a band-a-periodicity estimator for high-quality speech synthesis, Speech Communication, vol. 84, pp. 57-65, Nov. 2016. <http://www.sciencedirect.com/science/article/pii/S0167639316300413>
3. M. Morise: Implementation of sequential real-time waveform generator for high-quality vocoder, in Proc. APSIPA ASC 2020, pp. 821-825, Online, Dec. 7-10, 2020. <http://www.apsipa.org/proceedings/2020/pdfs/0000821.pdf>
4. J. de Langen: Playing and Recording Sound in Python, Real Python, 2022. <https://realpython.com/playing-and-recording-sound-python/>
5. P. Acsany: Logging in Python, Real Python, 2025. <https://realpython.com/python-logging/>
6. Python Software Foundation: Logging facility for Python (Standard Library Documentation), 2025. <https://docs.python.org/3/library/logging.html>
7. S. N. Kim: Python Logging: An In-Depth Tutorial, Toptal, 2023. <https://www.toptal.com/python/python-logging-best-practices>